



Python

University of Insubria

Introduction

Python

- is a great general-purpose programming language and writing programs is very fast (great for prototyping)
 - even if it is interpreted, so it runs slower than Rust or C++
- it has a huge collection of libraries
 - numpy, scikit-Learn, pandas, matplotlib, ..

Indentation is important!

Data Type

- numeric: int, float, long, ..

```
x = 3
print(x, type(x)) # 3 <class 'int'>
y = 2.5
print(y, type(y)) # 2.5 <class 'float'>
```

```
print(x + 1) # Addition;
print(x - 1) # Subtraction;
print(x * 2) # Multiplication;
print(x ** 2) # Exponentiation.
```

```
x += 1
print(x) # Prints "4"
x *= 2
print(x) # Prints "8"
```

- booleans: True, False

```
t, f, aa, bb = True, False, True, False # yes you can do this in Python
print(t, f, type(t)) # Prints "True False <class 'bool'>"
```

- string

```
hello = 'hello'
world = "world"
print(hello, len(hello)) # Prints "hello 5"
```

```
hw = hello + ' ' + world
print(hw) # hello world
```

```
hw12 = '%s %s! your number is: %d' % (hello, world, 12)
print(hw12) # hello world! your number is: 12
```

String operations

- replace

```
print(s.replace('l', '(ell)')) # Replace all instances of one substring with another;  
# prints "he(ell)(ell)o"
```

- substrings => 'in'

```
"ABC" in "This is the ABC of Python" True
```

- String comparison

```
'abc' < 'bbc' <= 'bbc'
```

True

```
'A' < 'a'
```

True

```
'1' < 'A'
```

True

String operations

- strings can be indexed with square brackets (starting from 0)

```
s = '123456789'  
print('First charcter of', s, 'is', s[0])  
print('Last charcter of', s, 'is', s[len(s)-1])
```

First charcter of 123456789 is 1
Last charcter of 123456789 is 9

- negative indices can also be used

```
print('First charcter of',s,'is',s[-len(s)])  
print('Last charcter of',s,'is',s[-1])
```

First charcter of 123456789 is 1
Last charcter of 123456789 is 9

String operations

substring (character range) can be specified with

- `[a:b]` to specify characters at indexes $\{a, a+1, \dots, b-1\}$

```
s = '123456789'
print("First three charcters", s[0:3])
print("Next three characters", s[3:6])
```

First three charcters 123
Next three characters 456

- an empty start and end of the range denotes the start/end of the string

```
print("First three characters", s[:3])
print("Last three characters", s[-3:])
```

First three characters 123
Last three characters 789

Operators

Symbol	Task Performed	Symbol	Task Performed	Operator	Meaning	Symbol	Task Performed
+	Addition	==	True, if it is equal	and	Logical and	&	Bitwise And
-	Subtraction	!=	True, if not equal to	or	Logical or		Bitwise OR
/	division	<	less than	"	"	^	Exclusive or
%	mod	>	greater than	not	Not	~	Negate
*	multiplication	<=	less than or equal to	"	"	>>	Right shift
//	floor division	>=	greater than or equal to	"	"	<<	Left shift
**	to the power of						

Lists

Lists in Python are easily resizable and can contain elements of different types

```
el = []           # Create an empty list
xs = [3, 1, 2]    # Create a List
print(xs, xs[2])  # prints "[3, 1, 2] 2"
print(xs[-1])     # Negative indices count from the end of the list; prints "2"

xs[2] = 'foo'     # Lists can contain elements of different types
print(xs)         # prints "[3, 1, 'foo']"

xs.append('bar')  # Add a new element to the end of the list
print(xs)        # prints "[3, 1, 'foo', 'bar']"

xs = xs + ['thing1', 'thing2'] # Adding lists (the += op works too)
print(xs)         # prints "[3, 1, 'foo', 'bar', 'thing1', 'thing2']"

removed = xs.pop() # Remove and return the last element of the list
print(removed, xs) # prints "'thing2', [3, 1, 'foo', 'bar', 'thing1']"
xs.append("thing2") # list.push() for symmetry with list.pop()
print(xs)         # prints "[3, 1, 'foo', 'bar', 'thing1', 'thing2']"
```


Slicing

python provides a concise syntax for accessing sublists; this is known as slicing

- `list[<start>:<stop>:<step>]`

```
nums = list(range(5)) # range is a built-in function
print(nums)           # prints "[0, 1, 2, 3, 4]"

print(nums[2:4])      # Get a slice from index 2 to 4 (exclusive); prints "[2, 3]"
print(nums[2:])       # Get a slice from index 2 to the end; prints "[2, 3, 4]"
print(nums[:2])       # Get a slice from the start to index 2 (exclusive); prints "[0, 1]"
print(nums[:])        # Get a slice of the whole list; prints "[0, 1, 2, 3, 4]"
print(nums[:-1])      # Slice indices can be negative; prints "[0, 1, 2, 3]"
nums[2:4] = [8, 9]    # Assign a new sublist to a slice
print(nums)           # Prints "[0, 1, 8, 9, 4]"
print(nums[-3:])      # prints "[8, 9, 4]"
print(nums[::-1])     # it starts from the end, towards the first, taking each element, prints [4, 9, 8, 1, 0]
```

Lists

```
num = [0,1,2,3,4,5,6,7,8,9]
print(len(num)) # prints "10"

print("min =",min(num)," max =",max(num)," total =",sum(num)) # prints "min = 0   max = 9   total = 45"

lst = [1,1,4,8,7]
lst.append(1)
print(lst) # prints "[1, 1, 4, 8, 7, 1]"

a = [1,1,1]
a.append([2,2])
print(a) # prints "[1, 1, 1, [2, 2]]"

lst.extend([10,11,12])
print(lst) # prints "[1, 1, 4, 8, 7, 1, 10, 11, 12]"

print(lst.count(1)) # prints "3"
```

if statement

```
if day == 'Monday':  
    print('Week-end is over!')  
elif day == 'Sunday' or day == 'Saturday':  
    print('Sleep!')  
else:  
    print('Meeeh')
```

Iterations

while

```
i = 0
while i < 3:
    print(i)
    i += 1
```

0
1
2

to iterate elements in a list

```
animals = ['cat', 'dog', 'monkey']
for animal in animals:
    print(animal)
```

cat
dog
monkey

Enumerate

to get the index of each element

```
animals = ['cat', 'dog', 'monkey']  
for idx, animal in enumerate(animals):  
    print('index: %d, item: %s' % (idx, animal))
```

```
index: 0, item: cat  
index: 1, item: dog  
index: 2, item: monkey
```

Zip

to iterate two or more lists in parallel

```
alist = ['a1', 'a2', 'a3']  
blist = ['b1', 'b2', 'b3']  
clist = ['c1', 'c2', 'c3']  
  
for a, b, c in zip(alist, blist, clist):  
    print(a, b, c)
```

```
a1 b1 c1  
a2 b2 c2  
a3 b3 c3
```

List comprehensions

```
nums = [0, 1, 2, 3, 4]
squares = []
for x in nums:
    squares.append(x ** 2)
print(squares)
```

[0, 1, 4, 9, 16]



```
nums = [0, 1, 2, 3, 4]
squares = [x ** 2 for x in nums]
print(squares)
```

[0, 1, 4, 9, 16]

```
nums = [0, 1, 2, 3, 4]
even_squares = [x ** 2 for x in nums if x % 2 == 0]
print(even_squares)
```

[0, 4, 16]

dictionary to store (key, value) pairs

[illegible]

Set

a set is an unordered collection of distinct elements

```
a = set(['a','a','b','b','b','b','b'])
print(a)                # prints "'a', 'b'"
animals = {'cat', 'dog'}
print(animals)           # prints "'cat', 'dog'"

print('cat' in animals)  # Check if an element is in a set; prints "True"
print('fish' in animals) # prints "False"

animals.add('fish')      # Add an element to a set
print('fish' in animals) # prints "True"
print(len(animals))      # Number of elements in a set; prints "3"

animals.add('cat')       # Adding an element that is already in the set does nothing
print(animals, len(animals)) # prints "'fish', 'cat', 'dog' 3"
animals.remove('cat')     # Remove an element from a set
print(animals, len(animals)) # prints "'fish', 'dog' 2"
```

Functions

Python functions are defined using the reserved keyword **def**

```
def sign(x):  
    if x > 0:  
        return 'positive'  
    elif x < 0:  
        return 'negative'  
    else:  
        return 'zero'
```

```
for x in [-1, 0, 1]:  
    print(sign(x))
```

negative
zero
positive

Functions

it is possible to indicate optional parameters

```
def hello(name, loud=False):  
    if loud:  
        print('HELLO, %s' % name.upper())  
    else:  
        print('Hello, %s!' % name)
```

```
hello('Bob')  
hello('Fred', loud=True)
```

```
Hello, Bob!  
HELLO, FRED
```

λ -constructors (inline functions)

lambda functions (anonymous functions) are inline functions that do not have a name

- *lambda arguments : expression*

```
# The statement creates an anonymous function with the Lambda keyword.  
# The x is a parameter that is passed to the lambda function.  
func = lambda x: x**x + 2  
  
print(type(func)) # prints "<class 'function'>"  
print(func(3))    # prints "29"
```

map

to perform functions on each element of a list

```
x = [1, 5, 3, 2, 7, 8, 3, 6]

print([func(elem) for elem in x])

# Similar to the MAP function:
# -----
# map functions expects a function object and any number of iterables
# like list, dictionary, etc.
# It executes the function_object for each element in the sequence and
# returns a list of the elements modified by the function object.
print(list(map(lambda x: x**x + 2, x))) # note the map() function

[3, 3127, 29, 6, 823545, 16777218, 29, 46658]
[3, 3127, 29, 6, 823545, 16777218, 29, 46658]
```

Classes and objects

```
class Greeter():  
    # Constructor  
    def __init__(self, name):  
        self.name = name # Create an instance variable  
  
    # Instance method  
    def greet(self, loud=False):  
        if loud:  
            print('HELLO, %s!' % self.name.upper())  
        else:  
            print('Hello, %s' % self.name)  
  
g = Greeter('Fred') # Construct an instance of the Greeter class  
g.greet()           # Call an instance method; prints "Hello, Fred"  
g.greet(loud=True)  # Call an instance method; prints "HELLO, FRED!"
```

Exceptions

```
def my_func(a):  
    return a * a  
  
try:  
    print(my_func("A"))  
except TypeError as e:  
    print('Caught exception:', e)
```

Caught exception: can't multiply sequence by non-int of type 'str'

Modules

a module allows to organize your code

import the whole module

- import math

```
import math  
print(math.sqrt(81))
```

9.0

```
import math as f  
print(f.sqrt(81))
```

9.0

import only specific functions

- from math import sqrt

```
from math import sqrt  
print(sqrt(81))
```

9.0

Acknowledgments

Part of this material has been adapted from the `CS231n` Python tutorial by Justin Johnson¹

(<http://cs231n.github.io/python-numpy-tutorial/>)

Code

Crivello di Eratostene

Crivello di Eratostene

Il modo più veloce di trovare tutti i numeri primi (minori di $\sim 10,000,000$) è quello di utilizzare il Crivello di Eratostene.

Il crivello produce una serie di numeri primi senza eseguire il test di primalità su ognuno.

Come funziona?

- Crea una lista di tutti i numeri positivi minori uguali ad n
- Cancella i multipli di tutti i numeri primi minori o uguali alla radice quadrata di n
- I numeri rimasti sono primi

Crivello di Eratostene

Esempio ($n = 30$)

- 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Il primo numero è 2, ed è primo. Cancelliamo tutti i multipli di 2. I numeri cancellati non sono primi.

- 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Il secondo numero è 3, ed è primo. Cancelliamo tutti i multipli di 3. I numeri cancellati non sono primi.

- 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Il terzo numero è 5, ed è primo. Cancelliamo tutti i multipli di 5. I numeri cancellati non sono primi.

- 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Il quarto numero è 7 ed è maggiore della radice quadrata di 30. Non esistono quindi multipli di 7 che non sono ancora stati cancellati. Il Crivello di Eratostene termina.

Crivello di Eratostene

```
1_python > erathostenes_sieve.py > ...
1  def erathostenes_sieve(n):
2      primes = [False, False] + [True] * (n - 2)
3      i = 2
4      while i < n:
5          if not primes[i]:
6              i += 1
7              continue
8              k = i * i
9              while k < n:
10                 primes[k] = False
11                 k += i
12             i += 1
13     return [i for i in range(2, n) if primes[i]]
14
15
16 primes = erathostenes_sieve(75)
17 print(primes)
18
```

```
● (3.9.18) simone@Mac cc % /Users/simone/.pyenv/versions/3.9.18/bin/python /Users/simone/erathostenes_sieve.py
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73]
○ (3.9.18) simone@Mac cc %
```

Functions

chr() and ord()

chr() function takes integer argument and return the string representing a character at that code point

- `x = chr(65)`
 - `print(x) => # A`

ord() function takes string argument of a single Unicode character and return its integer Unicode code point value

- `x = ord('A')`
 - `print(x) => # 65`

Bytes to Hex - Hex to Bytes

Converting bytes to hex

- `b'Hello World!'.hex()`
 - `# 48656c6c6f20576f726c64`

Converting hex to bytes

- `hex_str = "48656c6c6f20576f726c64"`
 - `bytes.fromhex(hex_str)`
 - `# b'Hello World'`

```
cc.py > ...
1  s = b"Hello World!".hex()
2  print(s)
3  # 48656c6c6f20576f726c6421
4
5  s = "48656c6c6f20576f726c6421"
6  res = bytes.fromhex(s)
7  print(res)
8  # b'Hello World!'
9
```


byteorder

byteorder determines the byte order used to represent the integer

- big endian
 - the most significant byte is at the beginning of the byte array
- little endian
 - the most significant byte is at the end of the byte array

Bytes to Int - Int to Bytes

Converting bytes to int

- `int.from_bytes(b'hello', byteorder='big')`
 - big: # 448378203247
 - little: # 478560413032

Converting int to bytes

- `num.to_bytes(2, byteorder='little')`
 - num = 1234
 - big: # b'\xd2\x04'
 - little: # b'\x04\xd2'

Bytes to Int

Converting bytes to int

- `int.from_bytes(b'hello', byteorder='big')`
 - big: # 448378203247
 - little: # 478560413032
- `int.from_bytes(b'\x00\x01', byteorder='little')`
 - big: # 1
 - little: # 256

Int to Bytes

Converting int to bytes

- `num.to_bytes(3, byteorder='little')`
 - `num = 1234`
 - `big: # b'\x00\x04\xd2'`
 - `little: # b'\xd2\x04\x00'`
- `num.to_bytes(4, byteorder='little')`
 - `num = 1234`
 - `big: # b'\x00\x00\x04\xd2'`
 - `little: # b'\xd2\x04\x00\x00'`

pwntools

pwntools is a set of tools that are widely used for CTF

- *“make exploitation during CTFs as painless as possible, and to make exploits as easy to read as possible”*

Installation

- `pip install pwn`

Documentation

- <https://docs.pwntools.com/en/stable/>

pwntools

remote

```
2  from pwn import remote
3
4  conn = remote('ftp.ubuntu.com',21)
5  res = conn.recvline()
6  print(res) # b'220 FTP server (vsftpd)\r\n'
7
8  res = conn.send(b'USER anonymous\r\n')
9
10 res = conn.recvuntil(b' ', drop=True)
11 print(res) # b'331'
12
13 res = conn.recvline()
14 print(res) # b'Please specify the password.\r\n'
15
16 conn.close()
17
```

pillow

PIL - Python Imaging Library

- provides extensive file format support, an efficient internal representation, and fairly powerful image processing capabilities

Installation

- `python3 -m pip install --upgrade Pillow`

Documentation

- <https://pillow.readthedocs.io/en/latest/installation/basic-installation.html>

printable

string.printable is a pre-initialized string used as a constant

- returns all digits, ascii_letters and whitespace

```
3 import string
4
5 print(string.printable)
6
```

```
0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```


PyCryptoDome

PyCryptoDome

PyCryptodome is a self-contained Python package of low-level cryptographic primitives.

Details

- Version: 3.21.0
- GitHub: <https://github.com/Legrandin/pycryptodome>
- Documentation: <https://www.pycryptodome.org/>
- Installation: *pip install pycryptodome*

PyCryptoDome

Modules

- **Crypto.Cipher:** Modules for protecting confidentiality that is, for encrypting and decrypting data (example: AES).
- **Crypto.Hash:** Modules for creating cryptographic digests (example: SHA-256).
- **Crypto.Util:** General purpose routines (example: prime number generator, XOR for byte strings).
- **Crypto.Random:** Modules for generating random data.

Crypto.Cipher

Modules for protecting confidentiality that is, for encrypting and decrypting data (example: AES)

API

```
>>> from Crypto.Cipher import DES
>>>
>>> key = b'-8B key-'
>>> cipher = DES.new(key, DES.MODE_OFB)
>>> plaintext = b'sona si latine loqueris '
>>> msg = cipher.iv + cipher.encrypt(plaintext)
```

Crypto.Hash

Modules for creating cryptographic digests (example: SHA-256).

- Cryptographic hash functions take arbitrary binary strings as input and return a fixed-length output of random type (called a digest or hash value)

API

```
>> from Crypto.Hash import SHA256
>>
>> hash_object = SHA256.new(data=b'First')
```

Crypto.Util

General purpose routines (example: prime number generator, XOR for byte strings).

- Padding:
 - `pad(data, block_size)`
 - `unpad(data, block_size)`
- `strxor`:
 - `strxor(t1, t2)`
- `number`:
 - `getPrime(N)`
 - `getRandomInteger(N)`
 - `inverse(u, v)`
 - `isPrime(N)`

Crypto.Random

Modules for generating random data

- `get_random_bytes()`
- submodule `random`
 - `getrandbits(N)`
 - `randint()`